Skriftlig eksamen i

Algoritmer og datastrukturer

8. januar, 2022

Dette dokument indeholder 4 sider, inklusive forsiden. Der er i alt 7 opgaver.

I programmeringsopgaverne skal du implementere i C++.

Opgave 1 (13 %)

Hvad er Store O (Big-Oh) tidskompleksiteten af func1? Begrund dit svar.

```
int func2 (int n) {
  int res = 0;
  for (int i = 0; i<n; i++) {
    res = res+1;
  }
  return res;
}

int func1(int n) {
  int x = 0;
  for (int i = 0; i<n; i++) {
    x = x+func2(n);
  }
  return x;
}
```

Opgave 2 (12 %)

Skriv en rekursiv funktion/metode med følgende signatur:

```
int sumDivisibleBy3(int N)
```

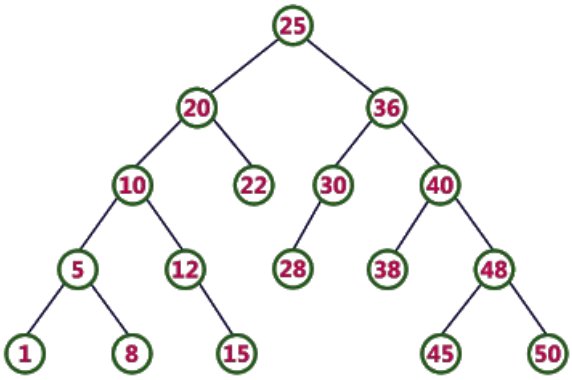
Algoritmen returnerer summen af positive heltal større end 0 og mindre end eller lig med N, som er dividerbare med 3. Du kan antage at `N>=0`.

Kaldt med `N = 12` er den korrekte returværdi 30 (3+6+9+12). Kaldt med `N = 14` er den korrekte returværdi også 30.

Du bedes optimere din metode/algoritme, således at overflødige rekursive kald undgås.

Opgave 3 (15 %)

Nedenstående figur er et binært søgetræ (binary search tree):



List den rækkefølge, som noderne bliver besøgt i, i en postorder og i en preorder traversering.
Hvad er summen af træets knudernes højde (internal path length)?
Er det et AVL-træ? Hvorfor eller hvorfor ikke?
Hvorledes vil træet se ud, når du har indsat et tal som svarer til 2*din alder (dvs. hvis du er 17 skal 34 indsættes). Noter din alder i besvarelsen af opgaven.

Opgave 4 (15 %)

Skriv en funktion/metode som kan finde median elementet i en samling af elementer. Elementerne er opbevaret i en `MaxHeap`. Du må KUN bruge funktioner/metoder på `MaxHeap`'en som har følgende:

```
bool isEmpty() const; //is the heap empty?
int size(); //number of elements in the heap
void insert(const Comparable& x); //add an element to the heap
const Comparable& findMax() const; // find the maximum element of the heap
void deleteMax(); //delete the maximum element of the heap
```

Argumentér for hvad tidskompleksiteten er for din løsning.

Opgave 5 (25 %)

Udvikl en effektiv algoritme med følgende signatur:

```
template<typename T>
vector<T> partitionEvenOdd(vector<T>& arg){
    //returnerer en vektor hvor alle lige elementer er før alle ulige elementer
}
```

Algoritmen må IKKE bruge ekstra lager (dvs du må ikke oprette ekstra `vector` variabler). Algoritmen skal have kompleksitet O(N), hvor N = |`arg`| - dette skal du argumentere for er tilfældet.

Opgave 6 (10 %)

En prioritetskø (priority queue) kan implementeres som et simpelt array af heltal, hvor det første element er ubrugt af hensyn til adressering. Kan det nedenstående array repræsentere en prioritetskø? Begrund dit svar.

{0,17,21,23,44,32,65,38,56,46,69,33,77,67,56,39,61,60,62,50,71}

Hvis det ikke kan repræsentere en prioritetskø, så modificer den så den kan og argumenter for hvorfor det er tilfældet

Opgave 7 (10 %)

Nedenstående tabel er en hashtabel, hvor der anvendes quadratic probing i tilfælde af kollisioner (collision resolution). Hash funktionen er $h(x) = x \bmod 11$. Følgende værdier er indsat: 22, 5, 16 og 27 (i denne rækkefølge):

Index	Value
0	22
1	
2	
3	
4	
5	5
6	16
7	
8	
9	27
10	

Positionerne (indexes) 1, 2, 3, 4, 7, 8 og 10 er ubrugte. Verificer at hashtabellen er korrekt og ret eventuelle fejl. Vis tabellens udseende efter at elementerne 1, 12, din alder samt dit studienummer er indsat og begrund hvorfor det indsættes hvor det indsættes samt forklar hvad det derudover sker. Noter i besvarelsen hvad din alder og studienummer er.

Skriftlig eksamen i

Algoritmer og datastrukturer

30. maj, 2022

Dette dokument indeholder 5 sider, inklusive forsiden. Der er i alt 6 opgaver.

I programmeringsopgaverne skal du implementere i C++.

Opgave 1 (13 %)

Hvad er Store O (Big-Oh) tidskompleksiteten af `func1`? Begrund dit svar.

```
static int myMethod( int N ){
  int x = 0; int y = 0;
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
      for (int k = 0; k < N*sqrt(N); k++) {
        x++;
      }
      j *= 2;
    }
    y = i;
  }
  for (int i = 0; i < N*N; i++) {
    y++;
  }
  return x+y;
}
```

Opgave 2 (12 %)

Skriv en **rekursiv** funktion/metode med følgende signatur:

`int oddSquare(int N)`

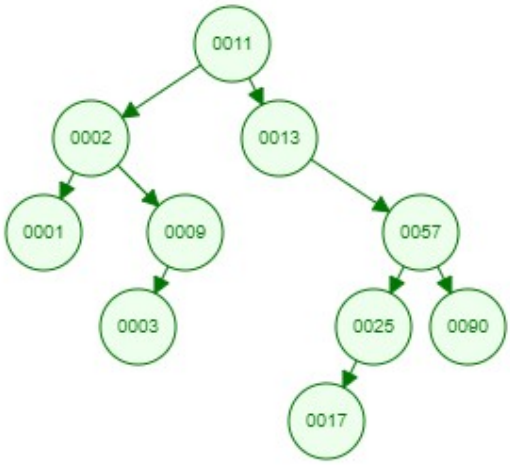
Algoritmen returnerer summen af de ulige tals kvadrattal fra 1 til N. Du kan antage at `N>=0`.

Kaldt med `N = 8` er den korrekte returværdi 84 ($1^2 + 3^2 + 5^2 + 7^2$). Kaldt med `N = 7` er den korrekte returværdi også 84.

Du bedes optimere din metode/algoritme, således at overflødige rekursive kald undgås.

Opgave 3 (20 %)

Nedenstående figur er et binært træ (binary tree)



1. Er det et binært søgetræ – hvorfor/hvorfor ikke?
2. List den rækkefølge, som noderne bliver besøgt i, i en inorder og i en preorder traversering.
3. Det er ikke et AVL-træ – hvorfor ikke? Hvad skal ændres for at det bliver et AVL træ?
4. Kan det have været et AVL træ inden den sidste AVL operation (indsættelse (`insert`) eller sletning (`delete`), men ikke rotation, som er en intern operation)? Eksempler på AVL operationer kunne være `insert(3)` eller `delete(12)`

Opgave 4 (25 %)

1: Skriv en klasse MaxHeap der kan indeholde `int`s ved brug af standardklassen `vector` fra C++ biblioteket `MaxHeap`'en skal have følgende operationer:

```
bool isEmpty() const; //is the heap empty?
int size(); //number of elements in the heap
void insert(const int x); //add an element to the heap
const int findMax() const; // find the maximum element of the heap
int deleteMax(); //delete and return the maximum element of the heap
```

2: Redegør for tidskompleksiteten af de fem operationer

`vector` har følgende operationer som du kan bruge (se næste side). Hvis du skal gennemløbe `vector`'en, skal du bruge en iterator. Iterator typen erklæres på følgende måde: `std::vector<int>::iterator` og har metoderne `begin()`, `end()`, `++` samt `*` (som virker som beskrevet i undervisningen)

Iterators

`begin` `cbegin` (C++11)	returns an iterator to the beginning (public member function)
`end` `cend` (C++11)	returns an iterator to the end (public member function)
`rbegin` `crbegin` (C++11)	returns a reverse iterator to the beginning (public member function)
`rend` `crend` (C++11)	returns a reverse iterator to the end (public member function)

Capacity

`empty`	checks whether the container is empty (public member function)
`size`	returns the number of elements (public member function)
`max_size`	returns the maximum possible number of elements (public member function)
`reserve`	reserves storage (public member function)
`capacity`	returns the number of elements that can be held in currently allocated storage (public member function)
`shrink_to_fit` (C++11)	reduces memory usage by freeing unused memory (public member function)

Modifiers

`clear`	clears the contents (public member function)
`insert`	inserts elements (public member function)
`emplace` (C++11)	constructs element in-place (public member function)
`erase`	erases elements (public member function)
`push_back`	adds an element to the end (public member function)
`emplace_back` (C++11)	constructs an element in-place at the end (public member function)
`pop_back`	removes the last element (public member function)
`resize`	changes the number of elements stored (public member function)
`swap`	swaps the contents (public member function)

The modifiers that require a parameter, takes an `Iterator` as argument

Opgave 5 (20 %)

Et (balanceret) binært søgetræ (binary search tree) kan implementeres som et simpelt array af heltal, hvor det første element (index 0) er ubrugt af hensyn til adressering. -1 angiver at elementet ikke findes/er tomt.

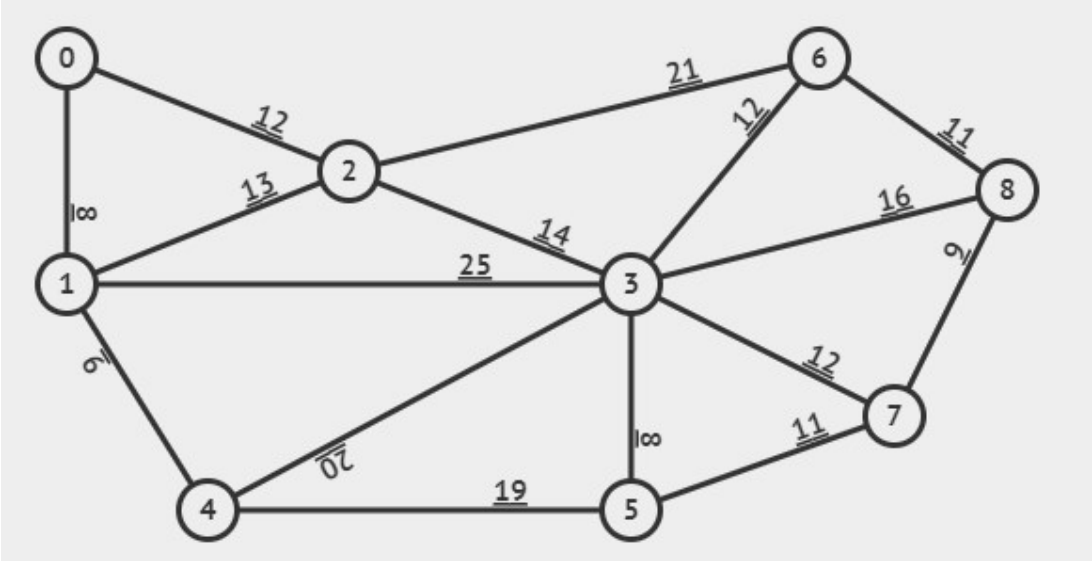
1. Beskriv hvorledes dette kan gøres effektivt (således at et givet element kan findes i O(log(N) forudsat det binære søgetræ er balanceret), hvor N er antallet af elementer i træet.
2. Kan det nedenstående array repræsentere et binært søgetræ? Begrund dit svar:

	27	21	50	17	23	48	-1	80	19	-1	-1	49	46	75	90
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

3. Hvis arrayet ikke kan repræsentere et binært søgetræ, så modificer det så det kan og argumenter for hvorfor det er tilfældet, samt tegn det binære søgetræ det repræsenterer
4. Hvilken træ gennemløbs algoritme kan bruges for at komme fra det binære træ til array implementeringen (dvs besøger knuderne i samme rækkefølge som de findes i arrayet)? Hvad er tidskompleksiteten af dette gennemløb?

Opgave 6 (10 %)

Givet følgende graf:



Hvorledes vil det mindste udspændende træ (minimum spanning tree) komme til at se ud hvis vi anvender Kruskal's algoritme. Beskriv den rækkefølge hvorved kanterne tilføjes.

Skriftlig reeksamen i

Algoritmer og datastrukturer

24. maj, 2023

Dette dokument indeholder 5 sider, inklusive forsiden. Der er i alt 5 opgaver.

I programmeringsopgaverne skal du implementere i C++.

Opgave 1 (20 %)

Betragt følgende funktion:

```
int main()
{
    int height = 0, line = 1, star = 0, spaces = 0;
    cout << "input a posetive number" << endl;
    cin >> height;
    if (height < 0) {
        cout << "you number was negative, try again" << endl;
        return 0;
    }
    while (height >= line) {
        spaces = 0;
        while (spaces < height - line) {
            cout << " ";
            spaces++;
        }
        star = 0;
        while ((line * 2 - 1) > star) {
            cout << "*";
            star++;
        }
        line++;
        cout << endl;
    }
    return 0;
}
```

Spørgsmål 1A: (5%)

Hvad er resultatet (hvad skrives ud) af at der bliver indtastet 3?

Spørgsmål 1b: (12%)

Hvad er tidskompleksiteten for (big-O) for programmet? Du skal argumentere for dit svar

Spørgsmål 1c: (25%)

Hvad er pladskompleksiteten (udtrykt i big-O)?

Opgave 2 (20 %)

Du skal lave en metode på en `BinaryHeap`

`changePriority(Comparable fromP, Comparable toP)`

som kan ændre prioriteten af et element/elementerne med prioritet `fromP` til at have prioritet `toP`. Du skal implementere den på `BinaryHeap` fra kurset (se slides og kode fra uge 7). Du må kalde de andre metoder på `BinaryHeap` i din implementation.

Opgave 3 (30 %)

Spørgsmål 3A: (25%)

Skriv en algoritme, der tager en string som input og returnerer det oftest forekommende ord i strengen. Strengen er almindelig tekst (små og store bogstaver), og ord adskilles med blanktegn, kommaer og punktum og kombinationer af disse (se eksempel nedenfor – du kan antage, at der altid er et mellemrum mellem to ord, og at det sidste tegn er altid et punktum). Ord betragtes som ens, selvom nogle forekomster begynder med et lille bogstav og andre med stort bogstav.

Du kan antage som en forudsætning for din algoritme, at inputparameteren kun kan indeholde små bogstaver [a-z] store bogstaver [A-Z], blanktegn, kommaer (,) og punktum (.).

Eksempel:

"*The cattle were running back and forth, but there was no wolf to be seen, heard or smelled, so the shepherd decided to take a little nap in a bed of grass and early summer flowers. Soon he was awakened by a sound he had never heard before*."

Det rigtige svar i eksemplet er ordet a, som forekommer tre gange.

Du må IKKE bruge

- metoder på `string`-klassen i C++
- `strtok()` eller lignende tokenizer metoder

Du må GERNE bruge de algoritmer og datastrukturer der er gennemgået i kurset

Mao du skal lave din algoritme ved brug af standard C++ samt kursets algoritmer og datastrukturer

Spørgsmål 3B (5%)

Angiv og rgumenter for tidskompleksiteten af din løsning

Opgave 4 (10 %)

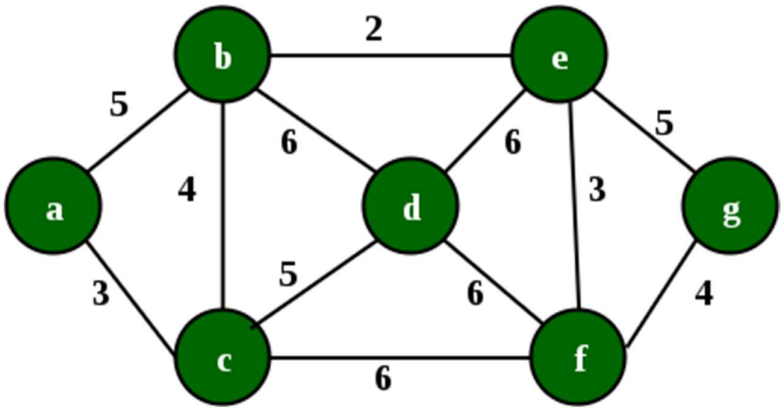
Betragt nedenstående hash-tabel

index	værdi
0	D
1	H
2	V
3	
4	
5	P
6	
7	X
8	E
9	
10	

Indeks 3, 4, 6, 9 og 10 er ledige. Hvad sker der, hvis et element, der hash'es til indeks 1, skal indsættes? Strategien, der bruges til kollisions håndtering, er quadratic probing.

Opgave 5 (20 %)

Betragt følgende graf:



Spørgsmål 5A (5%)
Hvor mange knuder og kanter vil der være i minimum spanning tree for ovenstående graf?

Spørgsmål 5B (15%)
Udregn et minimum spanning tree ved at benytte
a) Prim's algoritme. Algoritmen starter i kunde b
b) Kruskals algoritme.
Angiv for begge algoritmer i hvilken rækkefølge kanterne bliver inkluderet

Written Exam in Algorithms and Data Structures Fall 2024

This document contains ***9 pages***, including the front matter.

Response can be written in danish or english.

The exam set contains five questions which can be solved independently of each other.

Within each question there are multiple activities, with increasing order of difficulty.

Remember:

- Move on to other questions and activities if you are struggling with a particular activity. Come back to it later when you have time.
- When designing algorithms, an algorithm that returns an approximate solution is better than no algorithm.

Table of Contents

- Question 1 (20%): Algorithm Analysis
- Question 2 (15%): Recursion
- Question 3 (15%): Hashing
- Question 4 (15%): Algorithm Modification
- Question 5 (35%): Graph

Question 1 (20%): Algorithm Analysis

Consider the following algorithms:

```
bool alg1(const vector<int>& arr) {
    int count = 0;
    for (size_t i = 0; i < arr.size(); i++) {
        for (size_t j = i + 1; j < arr.size(); j++) {
            if (arr[i] == arr[j]) {
                count++;
            }
        }
    }
    return count > 0;
}
```

```
vector<int> alg2(const vector<int>& arr) {
    vector<int> prefixSums;
    int total = 0;
    for (int num : arr) {
        total += num;
        prefixSums.push_back(total);
    }
    return prefixSums;
}
```

```
void alg3(vector<int>& arr) {
    size_t n = arr.size();
    for (size_t i = 0; i < n; i++) {
        bool swapped = false;
        for (size_t j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Provide an answer to each the following, for each of the above algorithms:

1. Explain what the goal of the algorithm is (e.g., does it find the minimum element in the input vector?).
2. Write down the mathematical expression describing the worst case number of steps taken by the algorithm for a given `arr` of size n, also explaining what `arr`

should look like to trigger the worst case (e.g., `algX` takes $2 + 5n^2$ for `arr = {1, 1, 1, 1, ...}`).

3. What's the big-O worst case time complexity for this algorithm? If you have answered 2, you do not need to justify. Otherwise, provide a justification.

Question 2 (15%): Recursion

A **lonely child** in a binary tree is a node that has one parent and no siblings (i.e., the parent has only one child). The binary tree is represented by the following struct:

```
struct Node {
    int value;
    Node* left;
    Node* right;
};
```

Activities:

1. Write a **recursive function** to count the number of lonely children in the binary tree. The function should have the following signature:

   ```
   int countLonelyChildren(Node* root);
   ```

   - The function should take the root of the binary tree as input.
   - Return the number of lonely children in the binary tree.
2. Explain your solution, detailing what the base and recursive cases are in relation to searching the tree.
3. What is the worst case time complexity of your solution? Justify.

Question 3 (15%): Hashing

Consider the following hashing function:

$$h(c) = c * 31$$

where c is the integer-based key.

This function is used in the insertion of values into an hash table with linear probing of size $N = 7$:

Index	0	1	2	3	4	5	6
State	E	O	E	E	D	O	E

The state of each index is indicated by the letters:

- E: Empty
- O: Occupied (already filled)
- D: Deleted (previously used)

1. Show the state of the table after inserting an element with $c = 10$ with the hashing function h. Do not resize the table.
2. Show the state of the table after inserting an element with $c = 5$ into the hash map using the hashing function h. Start from the table obtained in question 1. Do not resize the table.
3. Is it possible that an element with $c = 13$ is stored in the hash table? Explain why or why not and which steps are needed to provide an exact answer.

Question 4 (15%): Algorithm Modification

Consider the following algorithm, already shown in question 1:

```
void alg3(vector<int>& arr) {
    size_t n = arr.size();
    for (size_t i = 0; i < n; i++) {
        bool swapped = false;
        for (size_t j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Activities:

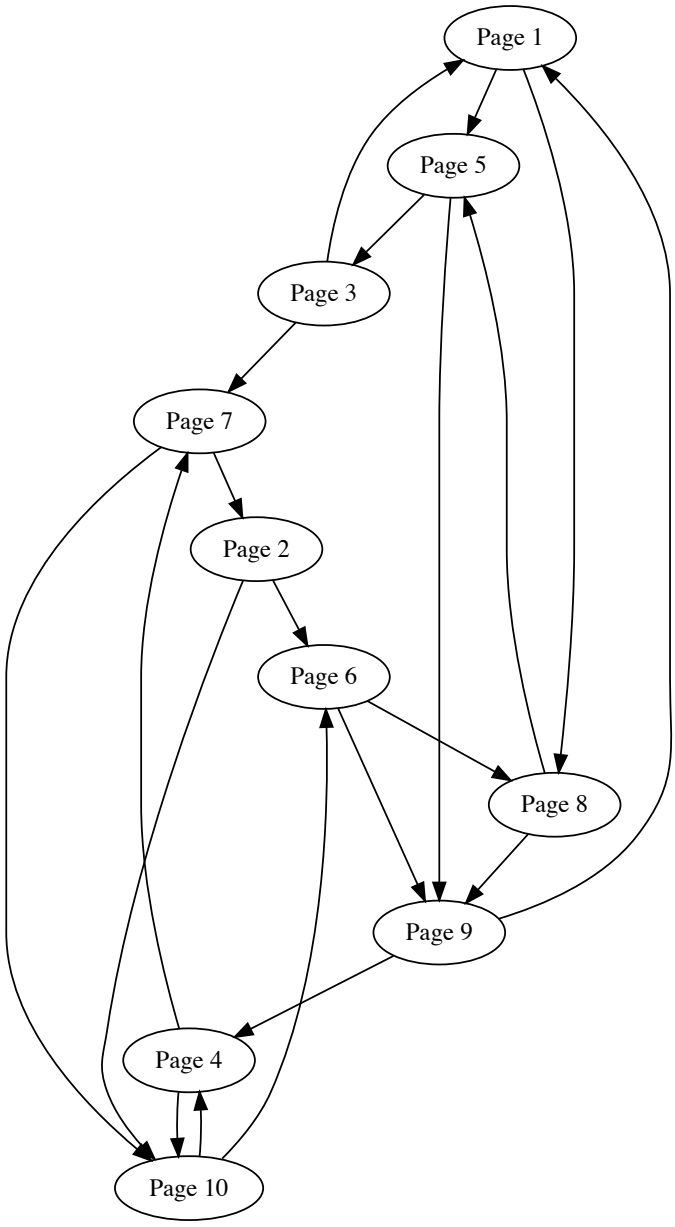
1. Illustrate the steps of the algorithm for `vector<int> arr = { 3, 1, 4, 1, 5, 9, 2 };`. Your answer should show the evolution of `arr`. For instance:

   ```
   { 3, 1, 4, 1, 5, 9, 2 } // Initial arr
   { 3, 1, 4, 5, 1, 9, 2 } // Something happened due to ...
   ...
   ```

2. Propose a modification to the algorithm `alg3` that stops when N elements have been placed in their final location in the array `arr`, and prints those N elements. The new signature should be `void alg3(vector<int>& arr, int N)` and you can assume that `N <= arr.size()`.

Question 5 (35%): Graph

A web search engine discovers, indexes, and ranks web content to deliver relevant results. Crawlers browse the internet, following links to gather webpage data. This content is indexed, extracting topics for quick retrieval. When a user submits a query, the engine matches it against the index using ranking algorithms based on relevance to deliver accurate results. The following graph is a small example of web pages linking to other web pages:



The figure illustrates that web page 1 is linked to by page 3 and page 9.

---

1. Using graph terminology, describe the characteristics of the graph
2. Which data structure would you use to represent the graph shown above: adjacency matrix or adjacency list? Argue for your answer
3. We wish to create a web crawler to traverse all the web pages. Which algorithm

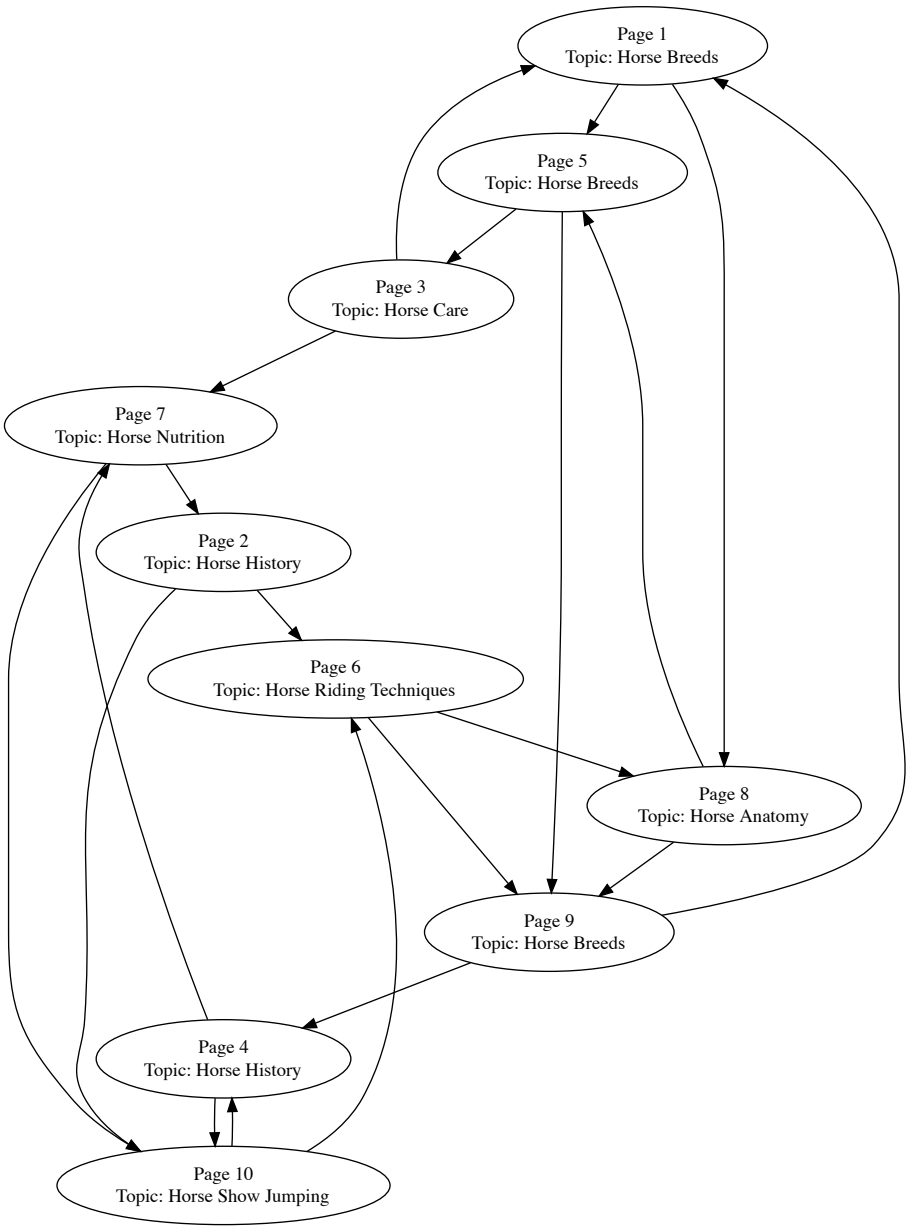
from the course is the most suitable for accomplishing this? Argue for your answer.

For our system, we define a page's rank by the number of pages that links to it. This implies that page 1 has a rank of two, since pages 3 and 9 link to it.

---

4. Suggest an algorithm to build a table of pages and ranking levels, that uses the graph representation you suggested in activity 2. Use pseudo code to describe the algorithm and also show the resulting ranking.

Each web page focus on a different topic as shown below and when searching for a topic, we wish to be presented with a list of pages concerning the desired topic that is ordered by rank. For instance, searching for "Horse History", should return the result [Page 4, Page 2], since Page 4 has a higher rank than Page 2.



Searching must be fast, while updating the underlying data can be slower. This is achieved by pre-processing topics and ranking of related pages.

---

1. To accomplish fast search of topics and keeping it easy to update the page

ranking, which data structure(s) from the course could be useful for storing topics and related ranked pages? Design an algorithm for searching and ranking results and provide an example of how it works, when searching for "Horse Breeds". Finally, you must specify the search time complexity of your solution.

Written re-exam in Algorithms and Data Structures Summer 2024

This document contains ***7 pages***, including the front matter.

Response can be written in Danish or English.

The exam set contains five questions which can be solved independently of each other.

Within each question there are multiple activities, with increasing order of difficulty.

Remember:

- Move on to other questions and activities if you are struggling with a particular activity. Come back to it later when you have time.
- For algorithms that are solving optimization problems, an algorithm that returns an approximate solution is better than no algorithm.

Table of Contents

- Question 1 (15%): Algorithm Analysis
- Question 2 (15%): Hashing
- Question 3 (15%): Search Tree
- Question 4 (20%): Algorithms for AVL trees
- Question 5 (35%): Graph

Question 1 (15%): Algorithm Analysis

Consider the following data structure and algorithm:

```
struct Node {
    int val;
    Node* left;
    Node* right;
    Node(int x){
        val = x;
        left = nullptr;
        right = nullptr;
    }
};

// Function to search a given key (Node.val)
Node* search(Node* root, int key) {
    while (root != nullptr) {
        if (root->val == key) {
            return root;
        }
        else if (key < root->val) {
            root = root->left;
        }
        else {
            root = root->right;
        }
    }
    return nullptr;
}
```

Nodes are connected to other nodes as either left or right node.

Provide an answer to each the following:

1. Is `search` a recursive or an iterative function? Explain why.
2. Argue what the worst-case time complexity (Big-O) of `search` is?
3. Describe the worse case scenarios for the complexity proposed above.

Question 2 (15%): Hashing

Consider the following hashing function:

```
unsigned long hash(const std::string& str) {
    unsigned long hash = 5381;
    for (char c : str) {
        hash = (hash * 33) + c;
    }
    return hash;
}
```

The value of the variable c of type char is the ascii value of the letter.

This function is used in the insertion of values into an hash table with linear probing of size N = 7:

Index	0	1	2	3	4	5	6
State	E	O	E	E	D	O	E

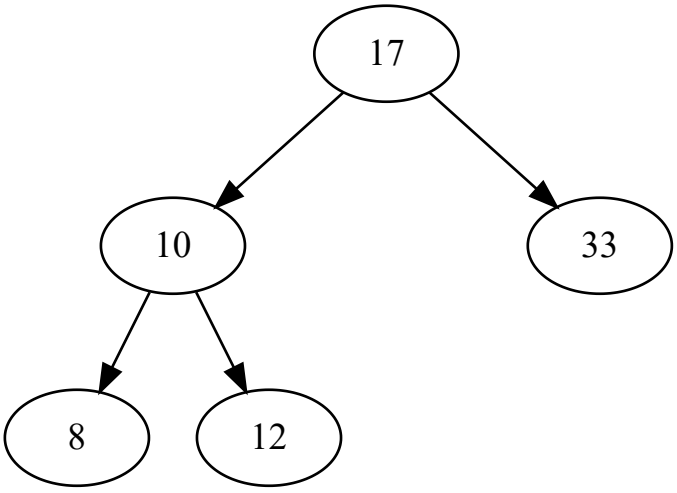
The state of each index is indicated by the letters:

- E: Empty
- O: Occupied (already filled)
- D: Deleted (previously used)

1. Show the steps to insert the string "go" into the hash map using the hashing function hash. Do not resize the table.
2. Show the steps to insert the string "ben" into the hash map using the hashing function hash. Do not resize the table.
3. Is it possible that the string "adam" is stored in the hash table? Explain why or why not and which steps are needed to provide an exact answer.

Question 3 (15%): Search Tree

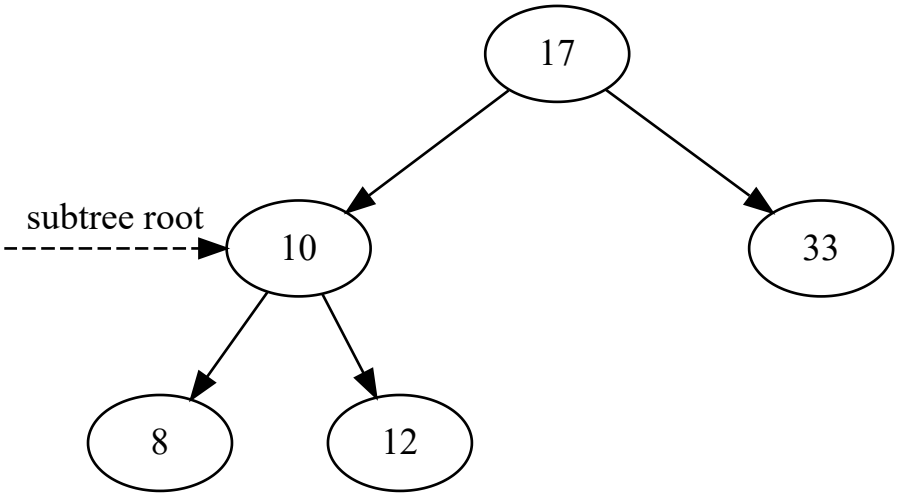
Given the following AVL Tree:



1. Argue why this tree has the properties of a valid binary search tree, more specifically an AVL tree.
2. Insert the value `14` and draw the associated rotation(s). **Note: This must be your own drawings!**
3. Explain why the worst-case time complexity (big-O) of searching an AVL tree is `log(N)`

Question 4 (20%): Algorithms for AVL trees

Given an AVL tree, imagine we want to create a recursive algorithm to find height of a given node's subtree.

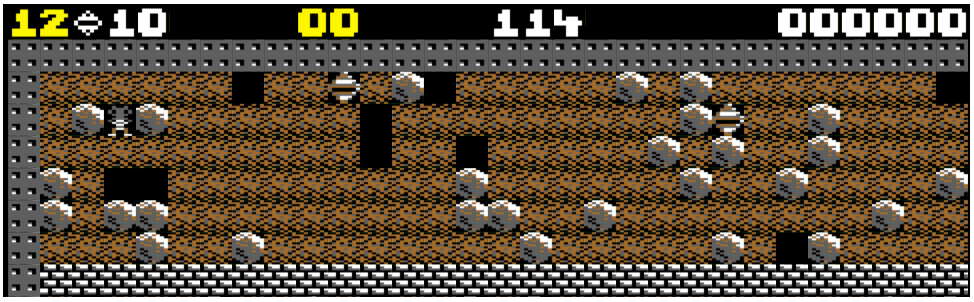


1. Describe in text what the base-case scenario(s) is(are), understood as case(s) that should cause the algorithm to stop its evolution/search and return.
2. Describe in text what the recursive-case scenario(s) is(are), understood as the case(s) that causes the algorithm to proceed with the next sub section of data.
3. Given the prototype for an AVL node shown below, write a function, `int height(AVLNode* subtreeRoot)` that returns the height of the subtree of a given node. The function must use a recursive approach, either invoking `height` itself or by using `height` to invoke a recursive function of your own produce.

```
struct AVLNode {
    int val;
    AVLNode* parent;
    AVLNode* left;
    AVLNode* right;
    AVLNode(int x){
        val = x; parent = nullptr;
        left = nullptr; right = nullptr;
    }
};
```

Question 5 (35%): Graph

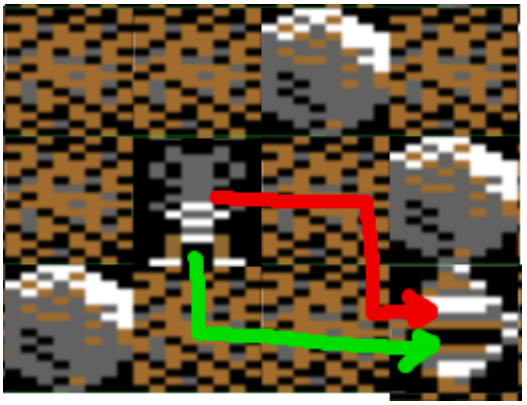
Boulder Dash is a classic arcade game. The player moves around a scenery with boulders and diamonds. The scenery is made up of equal-sized fields and the player can move up/down/left/right one field at a time. The player must collect diamonds and cannot walk through boulders.



1. How would you represent the game as a graph (i.e., which type of graph, which information you would assign to each node as a struct, and what does the adjacency matrix look like)? Which algorithm from the course is the fastest to find the cheapest way from start to a diamond? Justify your choice of graph and algorithm, explain how the algorithm can take into account the fact that the player cannot move through boulders. To help you justify your choices, draw the graph for the following scenario:

2. Suppose we introduce the following game rule:

> In boulder dash, if **you move down when standing underneath a boulder**, it will fall down and kill you and thus limit the ways you can move, as shown below (red = bad, green = ok).



Assume that you have to use Dijkstra's algorithm to reach the diamond (FindDijkstra() function shown in the slide 7 from lecture 10). How would you represent the game as a graph to take this new rule into account? Do you need to the modify the Dijkstra algorithm to take the new rule into account? If so, present the pseudo code with your modifications.

To justify your choice, show the first 3-4 steps that the Dijkstra algorithm (modified or not) would follow for the scenario shown in the previous activity, taking into account this new rule.

3. Each game level contains a number of diamonds that must be collected to finish the level. Propose an algorithm that finds the cheapest way to finish a level. Explain your algorithm in text **and** write the pseudo code.